

STMS UMR9912 - IRCAM
ENS DE LYON
UNIVERSITÉ CLAUDE BERNARD LYON 1

L3 - INFORMATIQUE
2 JUIN - 11 JUILLET 2025
STAGE

Jouabilité de suite d'accords pour guitaristes

Étudiant :
Robinson LANGLOIS

Encadrants :
Charlotte TRUCHET
Charles PRUD'HOMME

ircam
Centre
Pompidou



16 août 2025

Table des matières

1	Introduction	1
2	Préliminaires	1
2.1	Généralités sur la musique	1
2.2	Programmation par contraintes	3
3	Premier Modèle	5
3.1	Modélisation des objets	5
3.2	La distance sur le manche	5
3.3	Modèle PPC	5
4	Second modèle	7
4.1	Filtrage, contraintes en extensions et principe du MDD	7
4.2	Stratégie de recherche	9
4.3	SAC	11
4.4	Affiner le filtrage des accords	11
4.5	Minimisation et liberté laissée à l'instrumentiste	12
4.5.1	Personnalisation	12
4.5.2	Fonction de minimisation	13
4.6	Amélioration	14
4.6.1	Algorithmique	14
4.6.2	Préférences	15
5	Comparaison avec d'autres algorithmes similaires	16
6	Annexes	i
6.1	Notation Américaine	i
6.2	Exemple de transcription midi vers tablatures	i

1 Introduction

Dans la musique contemporaine, certaines partitions sont écrites par des personnes qui ne pratiquent pas l'instrument. Il peut arriver que certains compositeurs demandent à des musiciens de jouer des partitions difficiles, voire impossibles. Aujourd'hui, le problème peut aussi se poser avec l'arrivée d'outils de génération probabiliste de partitions, ou de suite de positions. C'est sur cette considération que Charlotte TRUCHET et Mikhail MALT ont décidés d'étudier la jouabilité de partition de guitare dans le cadre de la thèse de Charlotte TRUCHET et CODOGNET, "Musical constraint satisfaction problems solved with adaptive search". Le problème était le suivant : étant donné une suite d'accords, est-il possible de l'interpréter à la guitare, si oui, comment l'interpréter de la manière la plus facile ?

Au cours de mon stage, j'ai repris cette question dans l'objectif de réaliser un programme permettant de calculer une suite de positions à partir d'une suite d'accords en utilisant une approche par programmation sous contraintes. Pour l'implémentation de la résolution, j'ai utilisé le solveur Choco développé par Charles PRUD'HOMME PRUD'HOMME et FAGES, "Choco-solver : A Java library for constraint programming". Dans un premier temps, je me suis familiarisé avec la notion de programmation sous contraintes, et ensuite, j'ai réalisé un modèle pour résoudre le problème posé, en discutant avec des guitaristes (ne l'étant pas moi même) afin de laisser le plus de liberté et de maniabilité aux instrumentistes.

2 Préliminaires

Dans cette section, on définit ce que l'on utilisera par la suite pour la musique et pour la programmation par contraintes.

2.1 Généralités sur la musique

Dans l'introduction nous avons parlé d'accords, de chiffrage, de position, nous allons donc commencer par définir tout ceci correctement.

Definition 2.1 (*Note de musique*) *En musique, une note est un symbole ou une lettre permettant de représenter un fragment de musique par une convention d'écriture de la hauteur et de la durée d'un son.*

Dans ce problème où l'on s'intéresse à une suite d'accords, on ne considère que les hauteurs des notes, on reparlera cependant de la durée un peu plus tard. Pour décrire les hauteurs de notes, nous utiliserons l'encodage MIDI

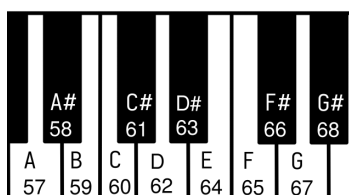


FIGURE 1 – Numérotations MIDI du centre du piano

Definition 2.2 (*Représentaion MIDI d'une note*) La représentation MIDI d'une note de musique est définie comme suit. Le do central du piano (en général 261.63Hz) est numéroté à 60, et le reste des notes est numéroté à $60 \pm \alpha$ où α est le décalage au do en nombre de demi ton (négatif si la note est plus basse). Ainsi, la valeur midi du la (de 440Hz) est 69. Dans la figure 1, on représente les numérotations MIDI des notes centrales du piano.

Nous définissons maintenant ce qu'est un accord.

Definition 2.3 (*Accord*) Un accord est un ensemble d'ensembles de notes, où chacun des éléments est construit en suivant des règles. Il est défini par sa fondamentale (la note de construction) et les intervalles, c'est à dire les écarts entre chaque note et la fondamentale. Les notes sont définies sans notions d'octaves et l'accord peut donc-être renversé ou avec des répétitions (voir 2).

Nous choisissons d'encoder les accords de la manière suivante :

$$Acc := (\mathbf{fonda}, u_0, \dots, u_n)$$

Où **fonda** est la note de construction de l'accord, et $\{u_i\}_{i \in \llbracket 0, n \rrbracket}$ sont les positions relatives de chacune des notes par rapport à **fonda** dans l'ordre croissant de taille des intervalles, en suivant la notation "américaine" (voir annexe 6.1).

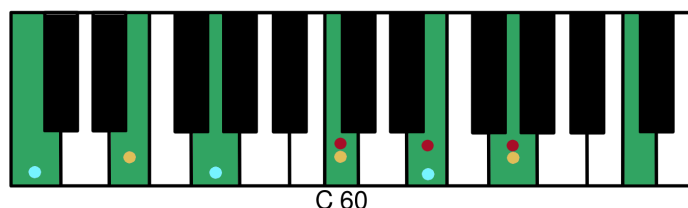


FIGURE 2 – Trois accords de Do majeur

Exemple 2.1 La figure 2 représente différentes manières de jouer l'accord de do majeur (noté CM) :

- les notes autorisées sont en vert.
- (60, 0, 4, 7) représenté en rouge.
- (60, 0, -8, 7) représenté en jaune.
- (48, 0, 16, 7) représenté en bleu.

Les trois accords sont des accords de do majeur. La liste n'est pas exhaustive.

Definition 2.4 (*Guitare*) On appellera par la suite guitare tout instrument qui possède un manche similaire à celui d'une guitare, les variables d'une guitare seront :

- Son nombre de cordes
- Sa scordature, (la note de chacune des cordes à vide)
- Son nombre de frettes (les petites barres présentes sur le manche délimitant les demis tons).

Pour toutes les guitares, augmenter d'une frette sur une corde augmente d'un demi-ton la note jouée.

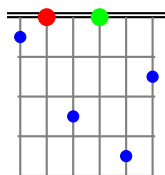
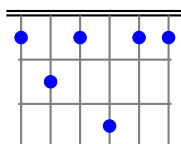


FIGURE 3 – Exemple de tablature.

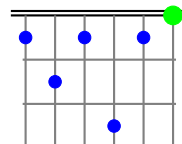
Ici, la guitare possède 6 cordes
et au moins 4 frettes.

Pour représenter une position sur le manche de la guitare, on utilise une tablature. Une tablature représente le manche de la guitare et les positions des doigts. Chaque point représente un point d'appui sur une corde en fonction de la frette. On représente comme dans la figure 3 une position par une tablature.

Le haut du manche est représenté par une double barre. Un point bleu représente un point de pression sur le manche (entre deux frettes sur une corde). Un point rouge représente une corde non jouée, c'est à dire que l'on ne pince pas la corde au moment de jouer l'accord. Un point vert représente une corde à vide, c'est une corde pincée au moment de jouer l'accord, mais sur laquelle on n'exerce pas de pression. On utilisera cette représentation dans la suite du rapport.



(a) Tablature avec barré



(b) Tablature avec barré partiel

FIGURE 4 – Tablatures avec barré

Une technique utilisée par les guitaristes est la technique du barré. Elle consiste à utiliser un doigt pour bloquer toute ou partie des cordes sur le même niveau. Par exemple, dans la représentation 4a on remarque que l'on ne peut pas utiliser 6 doigts pour bloquer chacune des six cordes mais l'on peut utiliser l'index pour bloquer toutes les cordes du manche en première frette et utiliser l'index et l'annulaire pour bloquer les cordes 2 et 4. Celles-ci sont par ailleurs bloquées par l'index en première frette mais cela n'est pas contradictoire car seul le point de pression le plus éloigné de la tête du manche compte. Cette technique requiert de la part de l'instrumentiste une maîtrise de son instrument. Elle peut être adaptée de sorte que le barré soit partiel, en ne commençant pas sur la première ou la dernière corde, ce qui permet de réaliser une position semblable à la tablature 4b.

2.2 Programmation par contraintes

Au cours de ce stage, on a eu une approche utilisant la programmation par contraintes. Nous donnons ici quelques définitions de base de ce paradigme. La programmation par contraintes (PPC) est un paradigme de programmation permettant de résoudre des problèmes combinatoirement grands. Dans ce paradigme, on sépare une partie modélisation avec le problème de satisfaction de contraintes, et une partie résolution, qui va utiliser les contraintes de problème pour réduire la taille de l'espace des solutions. Pour les définitions suivantes, on s'inspire de MARRIOTT et STUCKEY, *Programming with constraints : an introduction*.

Definition 2.5 (Modèle) Un modèle est un ensemble fini $\mathcal{X} = \{\mathcal{X}_i\}_{i \in I}$ (I fini) de variables indéterminées à valeurs dans un domaine $\mathcal{D} = \{\mathcal{D}_i\}_{i \in I}$, ainsi qu'un ensemble

fini $\mathcal{C} = \{\mathcal{C}_j\}_{j \in J}$ (J fini) de contraintes sur les variables. On le note :

$$\mathcal{M} = (\mathcal{X}, \mathcal{D}, \mathcal{C})$$

Definition 2.6 (*Solution*) Une solution d'un modèle est une instantiation des variables indéterminées dans leurs domaines tels que toutes les contraintes soient respectées.

Definition 2.7 (*Solver*) Un solver est un programme qui a pour entrée un modèle 2.5 et qui trouve toutes les solutions, ou une solution particulière 2.6 de ce modèle. Par la suite on utilisera le solver Choco PRUD'HOMME et FAGES, “Choco-solver : A Java library for constraint programming” pour modéliser notre problème et pour trouver les solutions qui nous intéressent.

Pour résoudre notre problème, nous allons utiliser la programmation par contraintes ?? . Pour ce faire, nous allons créer un modèle (définir des indéterminées, leurs domaines et des contraintes) dont les solutions soient les suites de positions les plus adaptées.

3 Premier Modèle

Je me suis d'abord familiarisé avec Java et la PPC en réalisant un premier modèle pour répondre à un premier problème :

A partir d'un accord, trouver toutes les positions jouables sur un manche de guitare.

Pour ce problème, nous modélisons dans un premier temps les objets étudiés (guitare, Guitariste, partition) puis nous décrivons des contraintes qui assurent une résolution du problème.

3.1 Modélisation des objets

On implémente une guitare par :

- **nc** : nombre de cordes (entier)
- **nf** : nombre de frettes (entier)
- **scorda** : scordature de la guitare (tableau d'entiers)

On implémente un ou une guitariste par :

- **tailleMain** : la taille de la main en le nombre de frêtes accessibles depuis le haut du manche (entier)
- **nbDoigtsUtilisés** : le nombre de doigts utilisés pour jouer (entier)
- **barréPossible** : -1 s'il est impossible de réaliser un barré, le nombre de frettes accessibles quand on fait un barré sinon (entier)

3.2 La distance sur le manche

Sur le manche de la guitare, les frettes ne sont pas disposées de manière régulière car le demi ton n'est pas défini de manière linéaire. Par exemple, pour augmenter d'une octave, on divise la longueur de la corde par deux. On peut alors trouver une relation entre le nombre de frettes accessibles depuis une position et **tailleMain**.

$$\text{distanceLégale}(\text{position}) = \text{tailleMain} \times 2^{\frac{\text{position}}{12}} \quad (3.1)$$

3.3 Modèle PPC

Nous définissons ici le premier modèle PPC réalisé.

Definition 3.1 (*Indéterminées*) Les indéterminées du modèle sont les frettes pressées sur chaque corde. Cela permet donc de définir une position $\mathcal{F} = \{\mathcal{F}_i\}_{i \in I}$ où I est une numérotation des cordes. Il y a donc **nc** indéterminées à valeur dans $\llbracket -1, \mathbf{nf} \rrbracket$. -1 représente une corde non jouée, sinon c'est une corde jouée et pressée sur la frette associée. 0 représente la corde à vide.

Definition 3.2 (*Contraintes*) Pour ce modèle, nous avons un ensemble de contraintes. Premièrement, il faut que les notes jouées soient uniquement des notes de l'accord. Cela nous amène à la première contrainte :

$$\forall i \in \llbracket 1, \mathbf{nc} \rrbracket, \exists j \geq 1, (\text{scorda}[i] + \mathcal{F}_i) - (\text{accord}[0] + \text{accord}[j]) = 0 \pmod{12} \quad (3.2)$$

Il faut aussi que l'ensemble des notes de l'accord soient jouées, cela nous amène à la deuxième contrainte :

$$\forall i \in \llbracket 1, \text{lenAccord} \rrbracket, \exists j \geq 1, (\text{scorda}[j] + \mathcal{F}_j) - (\text{accord}[0] + \text{accord}[i]) = 0 \pmod{12} \quad (3.3)$$

On rappelle que l'octave n'est pas importante et qu'il y a 12 demis tons dans une octave d'où le modulo 12.

Nous avons pour le moment des contraintes permettant d'assurer que ce que l'on joue correspond à la suite d'accords.

Il faut de plus modéliser la contrainte de distance pour avoir une première notion de jouabilité de la position. Il faut que la distance entre les doigts ne dépasse pas une certaine borne.

Après implémentation, le solveur choco trouve toutes les solutions du modèle demandé, donc toutes les positions qui respectent les contraintes pour un accord donné.

Nous avons donc créé un modèle de contraintes permettant, à partir du chiffrage d'un accord, de trouver l'ensemble des positions physiquement jouables de cet accord. Cependant la notion de physiquement jouable reste assez limitée. Et on s'est pour l'instant restreint à un seul accord.

4 Second modèle

En reprenant le modèle exposé en partie 3, nous changeons maintenant d'objectif. On souhaite à présent, à partir d'une suite d'accords, trouver une suite de positions physiquement jouables minimisant une certaine fonction de poids.

On a constaté deux choses, d'abord qu'il est assez fastidieux de déclarer des contraintes sur nos variables, d'autre part que l'ensemble des possibles parcouru par le solver est grand. Pour un accord, il est de nc^{nf} et donc pour une suite d'accords il est de $nbAccord^{nc^{nf}}$, cela fait beaucoup et nous allons raffiner l'ensemble des possibles.

Nous optons pour une approche en extension du problème. C'est à dire que l'on va filtrer les solutions possibles pour tendre vers une résolution plus efficace.

4.1 Filtrage, contraintes en extensions et principe du MDD

Le filtrage permet de restreindre l'ensemble de recherche. Par exemple, on peut rechercher une solution à notre problème dans l'ensemble des positions possibles (de cardinal nc^{nf}), ou dans l'ensemble des positions physiquement possibles (de cardinal $pos\phi$). C'est là le premier filtrage que l'on a effectué.

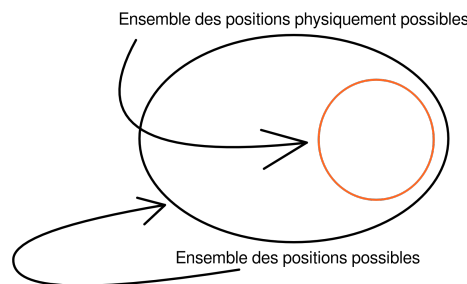


FIGURE 5 – Représentation du filtrage

Comme indiqué dans la figure 5, en parcourant l'ensemble des positions et en les filtrant nous allons passer d'un espace de recherche de $nbAccord^{nc^{nf}}$ à $nbAccord^{pos\phi}$.

Pour ce faire, on va réaliser une table des positions physiquement possibles.

Pour cela, on parcourt l'ensemble des positions et l'on vérifie pour chacune si elle est physiquement jouable voir code 6. Ici on a enrichi la notion de physiquement jouable, en effet, il n'est plus question que de distance entre les doigts mais aussi de la possibilité de réaliser des barrés ainsi que du nombre de doigts utilisés.

```

1 Tuples positions legales = new Tuples(true);
2 for (int i=0; i<(int)Math.pow((double)nf, (double)(nc+1)); i++)
3 {
4     int[] position_courante = new int[nc];
5     for (int j=0; j<nc; j++)
6     {
7         position_courante[j]=(i/((int)Math.pow((double)nf, (double)j))%nf
8         )-1;
9     }
10    if (est_legal(position_courante, nc, nf, distanceFirstFrette, barre
11    , nbDoigts))
12    {
13        positions legales.add(position_courante);
14    }
15 }

```

FIGURE 6 – Parcours de l'ensemble des positions et filtrage en fonction de la jouabilité

Une fois cette table établie, on va créer un modèle de contraintes où les indéterminées seront les suites de positions. L'avantage ici par rapport à la première partie est que le solver ne parcourt pas un arbre des possibles trop grand, mais qu'on calcule une fois un sous arbre pertinent. Pour ce calcul on parcourt effectivement l'ensemble des positions sur la guitare, mais d'une part on le réalise qu'une fois et non pas une fois par accords, et c'est un calcul beaucoup moins couteux que celui du solver.

Les tuples implémentés dans choco utilisent une structure efficace en temps et en espace : c'est la structure de MDD. On va un peu développer sur ce qu'est le MDD et montrer pourquoi c'est efficace.

Pour définir ce qu'est un MDD et pour montrer son intérêt, j'utilise un article de G.Perez et J.C.Régin PEREZ et RÉGIN, "Constructions and In-Place Operations for MDDs Based Constraints".

Definition 4.1 (MDD) *Un multi-valued decision diagram (MDD), est une structure de données qui représente des fonctions discrètes. Dans le cadre de la PPC, on utilise les MDD pour représenter des fonctions de la forme $f : \{0 \dots d - 1\}^r \rightarrow \{true, false\}$. La structure de données prend la forme d'un graphe composé de r couches. Les couches sont des ensembles d'arcs de même profondeur, et sur chacune des couches les noeuds possèdent au plus d arcs sortants allant à la couche d'après. La couche finale représente le noeud acceptant du MDD. Chaque arc les variables. Un tuple est accepté par un MDD ssi il existe dans le graphe un chemin de la première à la dernière couche tel que l'étiquette de l'arc à la couche i soit la i valeur du tuple.*

Voici un exemple pour illustrer la definition 4.1 :

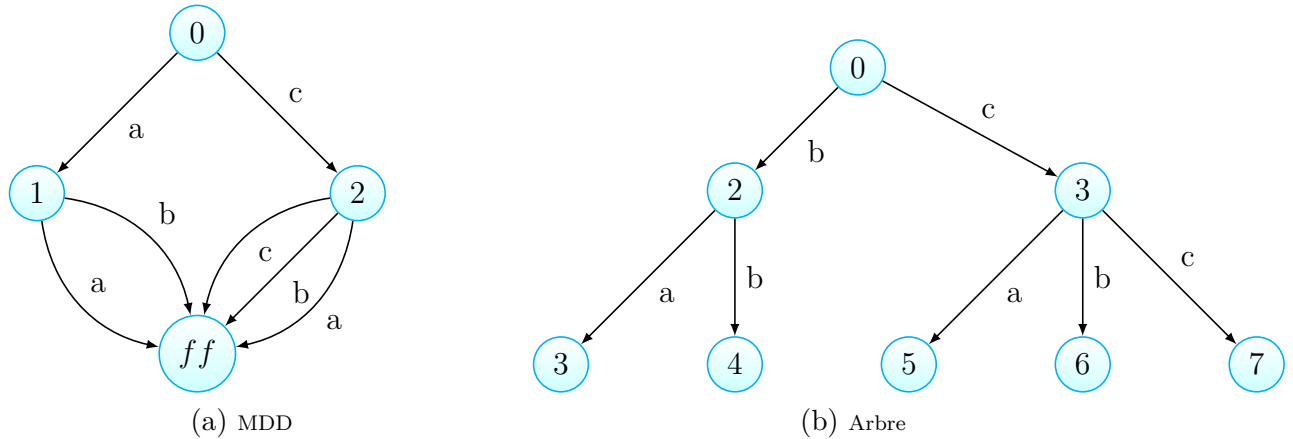


FIGURE 7 – Les deux graphes (MDD et arbre) représentent les tuples $\{\{a, a\}, \{a, b\}, \{c, a\}, \{c, b\}, \{c, c\}\}$

Les intérêts d'un MDD est qu'il est rapide de déterminer si un tuple appartient à la table qu'il décrit et qu'il prend peu de place en espace comparativement à un arbre. En effet, les sommets et arcs d'un MDD sont fusionnés de sorte à ce qu'il prenne le moins de place possible. De plus, la vérification de la validité d'un tuple se fait en la profondeur et donc linéairement en la taille du tuple. Les algorithmes pour transformer un ensemble de tuples en MDD sont déjà implémentés dans choco.

Une fois que l'on a créé la table on ajoute des contraintes de tables, c'est à dire que l'on réduit le domaine de la position aux positions physiquement jouables.

Dans la recherche de solution, on indique au solver une fonction à minimiser. Ainsi, choco va trouver une solution au problème qui est minimale pour la fonction indiquée. Nous dédions la sous partie 4.5.2 à cette fonction de minimisation qui prend en compte les attentes de l'instrumentiste.

On peut alors tester notre programme sur différentes suites d'accords. On observe que le programme fonctionne bien pour une suite de 4 accords, on obtient une solution en moins de 4 secondes, mais pour une suite de 6 accords l'algorithme met déjà plus longtemps ($\sim 20\text{sec}$), et pour 10 accords ($\geq 20\text{min}$) ce qui n'est pas acceptable. Il faut améliorer le temps de recherche.

4.2 Stratégie de recherche

Pour un parcours exhaustif, on a $\text{nbAccord}^{\text{pos}\phi}$ possibilités. Ce qui reste relativement grand.

Un première optimisation est de définir une stratégie de recherche dans le graphe des possibles. La stratégie de recherche consiste à indiquer au solver l'ordre des positions à fixer en premier. Elle permet d'éliminer des mauvais candidats et d'éviter de parcourir toutes les potentielles solutions. En effet, si l'on a trouvé une solution de poids p et qu'au cours de la recherche d'une solution (de la première position à la dernière position) on obtient un poids partiel $p' > p$ sur un accord intermédiaire, on stoppe la recherche pour ce candidat partiel.

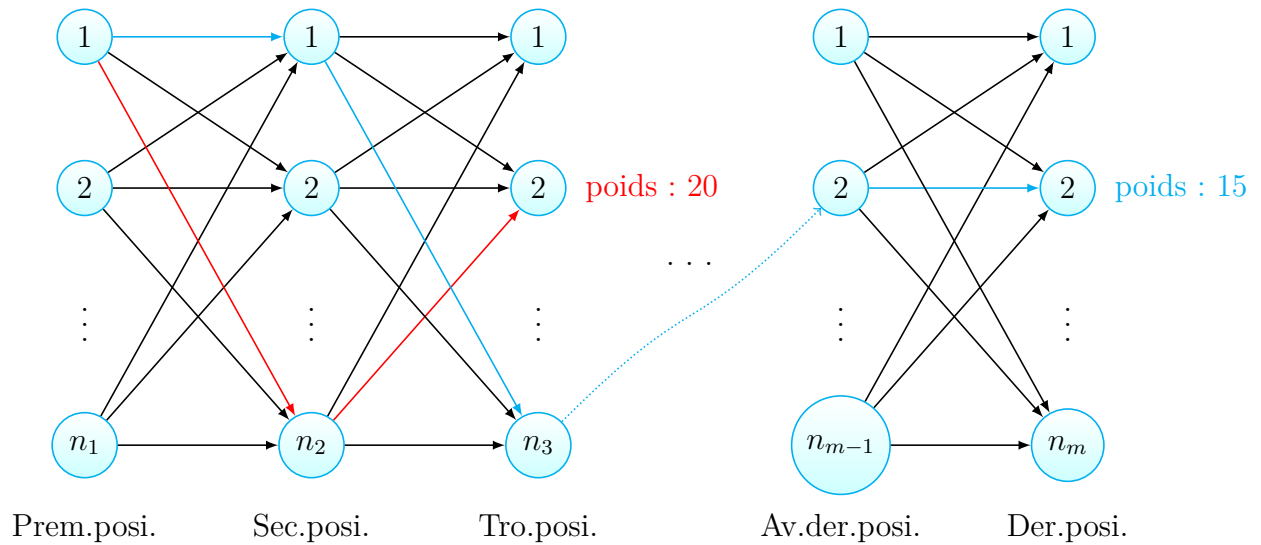


FIGURE 8 – Représentation d’élégage obtenu par la stratégie de recherche dans un ensemble de m positions avec n_i positions possibles pour la $i^{\text{ème}}$ indéterminée

Dans la figure 8, la stratégie de recherche va parcourir les positions possibles des tables de gauche (premier accord) à droite (dernier accord) en calculant le poids peu à peu et élimine les chemins de poids supérieurs à ceux déjà calculé. Par exemple, après avoir parcouru le chemin bleu et calculé son poids, le solveur parcourt, le chemin rouge et obtient en cours de parcours un poids supérieur au chemin bleu qui est complet. Sachant que le poids est une fonction croissante car dans la fonction de calcul on n’a pas de poids négatif, le solveur arrête la recherche après la section de chemin rouge. Cela permet de réaliser un élégage de l’ensemble de recherche.

Par exemple, on réalise le programme sans et avec une stratégie de résolution sur des entrées de tailles différentes avec comme fonction à minimiser une fonction qui est minimale quand les positions sont vers le haut du manche de la guitare (et donc que les positions sont proche de 0) :

Nombre d’accords	Sans	Avec
4	2s	2s
5	6s	3s
6	20s	4s
10	$\geq 20\text{min}$	38s

FIGURE 9 – Tableau comparatif du temps d’exécution en fonction du nombre d’accords de la suite avec et sans la stratégie de recherche

On peut voir dans le tableau 9 que la stratégie de recherche est très efficace. Mais cela s’explique en partie par l’objectif de minimisation et par la suite d’accords utilisés (ici les 4 premiers accords de Let It Be joués en boucle). La stratégie de recherche peut s’avérer plus ou moins efficace en fonction de la suite d’accords.

4.3 SAC

Une méthode d'optimisation par prétraitement conseillée par Charles est celle de la SAC (singleton arc consistency). Charles a développé une classe choco permettant d'appliquer quelques stratégies de prétraitement que j'ai pu tester dans le cadre de mon stage. Nous allons détailler ce qu'est la SAC et comment le processus permet au programme de gagner en efficacité. Pour définir le principe de SAC, on s'appuie sur le chapitre BESSIERE, "Chapter 3 - Constraint Propagation".

On pose

$$N = (\mathcal{X}, \mathcal{D}, \mathcal{C})$$

une instance du problème.

Definition 4.2 (*Consistance*) Soit $v_i \in \mathcal{D}(x_i)$, v_i est consistante avec $c \in \mathcal{C}$ ssi il existe une valuation τ vérifiant c tel que $\tau[x_i] = v_i$.

Definition 4.3 (*Arc-Consistant*) Soit $c \in \mathcal{C}$, $\mathcal{D}$ est arc-consistant sur c pour $x_i \in \mathcal{X}$ ssi pour tout $v_i \in \mathcal{D}(x_i)$, v_i est consistante avec c dans $\mathcal{D}$. On dit de N qu'il est arc-consistant ssi pour tout $x \in \mathcal{X}$, $\mathcal{D}$ est arc-consistant pour x sur toute contrainte c de $\mathcal{C}$.

Definition 4.4 (*Arc-Inconsistant*) N est arc-inconsistant ssi l'ensemble vide est l'unique domaine strictement plus petit que $\mathcal{D}$ qui est arc-consistant pour toutes variables, sur toutes contraintes.

Definition 4.5 (*Singleton Arc Consistant*) N est singleton arc consistant (SAC) ssi pour toute indéterminée $x_i \in \mathcal{X}$ et pour toute valeur $v_i \in \mathcal{D}(x_i)$, le sous problème $N|_{x_i=v_i}$ n'est pas arc inconsistant.

Un modèle qui est singleton arc consistant est un modèle pour lequel les domaines sont minimaux, dans le sens où il n'existe pas de valuations qui ne vérifie pas l'ensemble des contraintes. Transformer un modèle en modèle SAC permet donc de réduire ses domaines et d'avoir un espace de recherche restreint. Par exemple, transformer le modèle étudié dans cette partie en modèle SAC garantit que le domaine de l'indéterminée de la première position comporte uniquement des positions du premier accord. Sans la SAC, si le premier accord est un do majeur (do, mi, sol), une position avec la deuxième corde à vide (la) aurait été présente dans les domaines, alors qu'avec la sac, une position dans laquelle un la est joué va à l'encontre de la contrainte d'accord et n'apparaît pas dans les domaines. Ainsi, en réalisant un préprocessing, le solver choco raffine les domaines. Une fois le préprocessing effectué, la résolution du problème est beaucoup plus rapide. Il y a certes un temps de préprocessing mais il n'impacte pas le gain de temps pour la résolution.

4.4 Affiner le filtrage des accords

Après discussion avec Charlotte et Charles, nous avons conclu que si la SAC est performante, c'est que l'ensemble des domaines peut-être mieux décrit dans la modélisation du problème.

On va donc améliorer la fonction de filtrage. Au lieu de filtrer toutes les positions physiquement jouables, on va créer une table de hashage qui à chaque accord de la suite

associe un MDD représentant un ensemble de positions autorisées. Lors du filtrage, à chaque position calculée, si elle est physiquement jouable et si elle correspond à un accord de la suite étudiée, on l'ajoute dans le MDD associé à cet accord. On obtient alors autant de tables que d'accords différents dans lesquels sont stockés uniquement les accords physiquement jouables du bon accord.

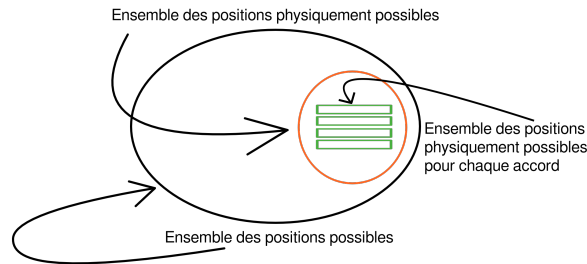


FIGURE 10 – Représentation de l'affinage du filtrage prenant en compte la nature des accords.

Comme représenté dans la figure 10, on obtient un plus grand nombre de tables mais toutes restreintes à l'essentiel. En effet, elles comportent uniquement les positions pertinentes. On aura alors un espace de recherche de taille $\prod_{i=0}^{nbAccord} |table.get(accord[i])|$. Finalement, compte tenu de la taille des tables, la taille de l'espace de recherche est très inférieure à celle de l'ensemble des positions physiquement jouables.

Une fois cela implémenté, on obtient un algorithme plus performant. On observe qu'en ajoutant le processus de transformation en SAC, on ne gagne pas en rapidité. Le prétraitement s'effectue très rapidement. On en conclut qu'il n'y a que très peu de domaines raffinés par le pré-processing SAC dans cette modélisation du problème et qu'il n'est donc plus nécessaire. D'une certaine manière, on a réalisé le travail du prétraitement de la SAC au moment du filtrage. Cela revient à dire que l'on a modélisé le problème de manière à avoir les domaines les plus petits possibles.

Dans cette modélisation, les seules réelles contraintes indiquées au solver sont des contraintes d'appartenance des positions à une table, et une contrainte de minimisation. Les contraintes de correspondance position / accord sont inutiles car toutes les positions du domaine sont des positions correspondant à l'accord demandé.

4.5 Minimisation et liberté laissée à l'instrumentiste

Le but de ce projet peut être vu de deux manières. D'une part, permettre à des personnes pour qui la guitare n'est pas l'instrument de prédilection notamment des débutants de trouver une suite de doigtés pour une suite d'accords. Mais d'un autre côté certains instrumentistes peuvent aussi vouloir utiliser cet outil pour trouver des doigtés en fonction de contraintes qu'eux mêmes poseraient. C'est pourquoi, avec l'aide de deux guitaristes, Mikahil Malt et Cléo Langlois, nous avons réfléchi à comment donner de la liberté à une personne utilisant cet outil.

4.5.1 Personnalisation

Nous avons donc réalisé une section préférence dans la classe définissant l'instrumentiste. Dans cette section, on peut renseigner différents critères :

- Avoir les positions les plus proches d’une certaine frette
- Jouer la note fondamentale de l’accord à la basse
- Avoir le moins de cordes non jouées
- Avoir le moins de cordes à vide car le son d’une telle corde est moins joli
- Avoir une distance minimale entre les positions adjacentes temporellement
- Eviter de jouer deux fois des notes de même fréquence (dans le cas où il y aurait un désaccordage même léger de l’instrument, il peut y avoir un problème de phase entre ces différents sons).

Pour chaque critère, soit il est fixé et on élimine dans le filtrage toutes les positions qui ne répondent pas au critère qui est exprimé avec un système de bornes, soit il est laissé à la minimisation dans ce cas avec un poids associé. On peut alors définir des profils, par exemple celui d’une personne débutante qui préférera rester en haut du manche, avoir le plus de cordes à vide et changer de position le moins possible. On peut aussi laisser libre cours à l’imagination de l’instrumentiste.

4.5.2 Fonction de minimisation

La fonction de minimisation est une fonction qui renvoie une indéterminé et qui traite d’indéterminées. Il se trouve que faire des manipulations arithmétiques et des tests sur les valeurs des indéterminées c’est assez compliqué. Par exemple, une fonction que j’ai beaucoup utilisée est la suivante :

$$f : \begin{cases} \mathbb{Z} & \longrightarrow \{0, 1\} \\ n & \longmapsto \frac{1 + \frac{n - |n-1|}{2}}{2} \end{cases}$$

Cette fonction renvoie les entiers strictement positifs sur 1 et les entiers négatifs sur 0. Avec des décalages et des compositions d’elle même, f permet par exemple de compter le nombre de cordes non jouées, de le pondérer avec le poids associé et de l’ajouter à la quantité à minimiser. Dans mon code j’ai appelé cette fonction `surjectionPositif`.

```

1 if (!player.preferences.cordesNonJoueesFix)
2 {
3   for (int i=0; i<doigtes.length; i++)
4   {
5     for (int j=0; j<doigtes[i].length; j++)
6     {
7       res = res.add(((ClasseOutilsMaths.surjectionPositif(model,
8         doigtes[i][j].add(1).intVar()).sub(1)).mul(-1).mul(player.
9         preferences.cordesNonJoueesPoids)).intVar());
10    }
11  }
12 }
```

FIGURE 11 – Exemple d’utilisation de `surjectionPositif` afin de compter le nombre de cordes non jouées dans la position

Dans le code 11, chaque corde non jouée dans la position `i` provoque un incrément de la quantité à minimiser de `player.preferences.cordesNonJoueesPoids` qui est

le poids associé à la minimisation du nombre de cordes non jouées défini dans les préférences.

On a construit la fonction à minimiser comme suit :

```
1 static IntVar fonctionToMinimize(Model model, IntVar[][] doigts, int
    nf, int[][] chords, int[] scordature, int[][] parametres)
```

Dans la fonction on rentre toutes les informations nécessaires : modèle, les indéterminées, le nombre de frettes, les accords ...

Mais on indique aussi des parametres qui vont pouvoir modifier la fonction de minimisation.

Les paramètres de préférence sont définis dans la classe `preference`. Ils fonctionnent de la sorte :

Chaque paramètre listé dans la section 4.5.1 est défini par plusieurs variables : une variable booléenne définissant si la contrainte est à fixer ou à minimiser, puis une variable de bornes à utiliser en cas de contraintes fixes, et un poids et des limites dans le cas de la contrainte de minimisation.

Voici un tableau listant les paramètres laissés à l'instrumentiste.

Fix	Borne	Limite	Poids
fretteMoyenneFix	fretteMoyenneSup fretteMoyenneInf	fretteMoyenne	fretteMoyennePoids
fondaBassFix	/	/	fondaBassPoids
cordesNonJoueesFix	cordesNonJoueesMax	/	cordesNonJoueesMax
pasRepetitionFix	pasRepetitionMax	/	pasRepetitionPoids
cordesAVidesFix	cordesAVidesMax	/	cordesAVidesPoids

FIGURE 12 – Tableau des paramètres ajustables par l'instrumentiste

On peut ainsi modifier la recherche dans la fonction à minimiser, en fonction des choix de l'instrumentiste.

4.6 Amélioration

Avec mes encadrants et Mikahil Malt, nous avons considéré plusieurs pistes d'amélioration à ce second modèle. D'une part, il peut y avoir des améliorations sur le plan algorithmique et l'efficacité du programme, et de l'autre, il peut y avoir des améliorations sur le plan des préférences laissées au guitariste.

4.6.1 Algorithmique

La fonction à minimiser est une fonction qui prend en entrée des `IntVar` et qui renvoie un `IntVar`. Bien que cela soit possible, traiter des `IntVar` de manière arithmétique est assez fastidieux. Ainsi, la manière de calculer la fonction de minimisation n'est pas optimale. Une manière d'améliorer cela serait d'ajouter à l'étape du filtrage des positions le stockage d'informations par rapport à la position. On pourrait stocker toutes les informations nécessaires à la fonction de minimisation, comme le nombre de cordes à vide. Au cours de la fonction de minimisation, on n'aurait plus qu'à récupérer ces informations déjà calculées au lieu d'utiliser des expressions arithmétiques sur des

indéterminées. J'ai commencé à implémenter un tel système mais je n'ai pas pu le finaliser.

4.6.2 Préférences

Avec Mikhaïl Malt et Charlotte Truchet, nous avons réfléchi à une manière de déterminer la position précise des doigts sur le manche. Cela permettrait d'indiquer sur la tablature quel doigt poser au lieu d'indiquer où les cordes doivent être bloquées. On pourrait aussi ajouter de la libérer à l'instrumentiste dans les préférences ainsi qu'un confort de lecture. On pourrait alors avoir une manière plus précise pour déterminer le barré ainsi qu'une distance entre les doigts plus fine avec la position exacte du doigt. On peut déterminer la position des doigts en considérant l'ordre suivant :

- Un premier ordre de l'index à l'auriculaire sur les frettes décroissantes.
- Un second ordre des cordes graves aux cordes aigües sur la même frette.

On peut alors déterminer une position précise des doigts sur le manche, mais cela reste à implémenter.

On peut aussi imaginer définir un ensemble de préférences (un par accord) pour adapter les préférences au cours du morceau.

On pourrait définir une durée entre chaque accord, ce qui permettrait par exemple entre deux accords distants d'une longue période temporelle de sectionner la partition et d'opérer le calcul sur des sous suites d'accords en considérant que l'instrumentiste peut pendant le laps de temps changer sereinement de position. Les deux ajouts précédents pourraient, combinés, permettre de définir un modèle de résolution pour trouver les positions pour une partition et non plus une suite d'accords.

On pourrait aussi réfléchir à une meilleure modélisation des doigts, vu non pas de manière abstraite mais basée sur la morphologie des doigts et de la main. En reprenant le travail de Leandro Costalonga COSTALONGA, "Biomechanical modelling of musical performance : a case study of the guitar", on pourrait déterminer de manière plus fiable si une position est jouable ou si elle ne l'est pas. On pourrait aussi ajouter une dimension de fatigue ou de difficulté musculaire pour le jeu d'un morceau.

5 Comparaison avec d'autres algorithmes similaires

Il existe déjà des logiciels qui permettent à partir d'une partition, ou même d'un fichier audio de déterminer une partition ou des tablatures. Ces outils, comme celui présenté dans HAMBERGER et al., *Fretting-Transformer : Encoder-Decoder Model for MIDI to Tablature Transcription* qui transforme une partition MIDI en tablatures, utilisent le modèle de LLM T5 de réseaux neuronaux. Au cours d'une discussion, Mikhaïl Malt a soutenu que les modèles utilisant des réseaux neuronaux sont certes très efficaces mais ne permettent pas à l'instrumentiste d'apprendre. En effet, si le modèle a été entraîné sur un corpus de tablatures, l'instrumentiste ne se pose pas de question à l'utilisation du logiciel. En revanche, l'approche par programmation par contrainte présente un intérêt pédagogique qu'on ne trouve pas dans les approches probabilistes. Une personne qui utilise le modèle présenté dans ce rapport doit s'interroger sur la manière de jouer de la guitare. Quels sont les paramètres à ajuster et comment ? Cela permet alors aux instrumentistes chevronnés de s'interroger sur des automatismes qu'ils ont acquis lors de leur apprentissage. Tandis qu'un modèle probabilistes va tendre à conforter l'instrumentiste dans des habitudes communes aux guitaristes. Mikhaïl a aussi insisté sur l'intérêt de la minimalité et de la démonstration de la solution apportée par le modèle de programmation sous contraintes qui n'est pas présente dans les modèles probabilistes.

Références

Bessiere : Chapter 3 - Constraint Propagation

BESSIEREsac

Christian BESSIERE. “Chapter 3 - Constraint Propagation”. In : *Handbook of Constraint Programming*. Sous la dir. de Francesca ROSSI, Peter VAN BEEK et Toby WALSH. T. 2. Foundations of Artificial Intelligence. Elsevier, 2006, p. 29-83. DOI : [https://doi.org/10.1016/S1574-6526\(06\)80007-6](https://doi.org/10.1016/S1574-6526(06)80007-6). URL : <https://www.sciencedirect.com/science/article/pii/S1574652606800076>.

Rés. : Publisher Summary This chapter discusses that constraint reasoning involves various types of techniques to tackle the inherent intractability of the problem of satisfying a set of constraints. Constraint propagation is one of those types of techniques. It is central to the process of solving a constraint problem and indispensable for constraint reasoning. The chapter examines that constraint propagation embeds any reasoning, which consists in explicitly forbidding values or combinations of values for some variables of a problem, because a given subset of its constraints cannot be satisfied otherwise. It is in constraint reasoning that this concept shows its most accomplished form. There is no other field in which the concept of constraint propagation appears in such a variety of forms, and in which its characteristics have been analyzed. The chapter reviews that it formalizes all constraint propagation approaches within a unifying framework and discusses the main existing types of constraint propagation.

Costalonga : Biomechanical modelling of musical performance : a case study of the guitar

biomechanicalCostalonga

Leandro COSTALONGA. “Biomechanical modelling of musical performance : a case study of the guitar”. Thèse de doct. Citeseer, 2009.

Hamberger et al. : Fretting-Transformer : Encoder-Decoder Model for MIDI to Tablature Transcription

outilIA

Anna HAMBERGER et al. *Fretting-Transformer : Encoder-Decoder Model for MIDI to Tablature Transcription*. 2025. arXiv : 2506.14223 [cs.SD]. URL : <https://arxiv.org/abs/2506.14223>.

Marriott et al. : Programming with constraints : an introduction

introPPC

Kim MARRIOTT et Peter J STUCKEY. *Programming with constraints : an introduction*. MIT press, 1998.

Perez et al. : Constructions and In-Place Operations for MDDs Based Constraints

PerezR16

Guillaume PEREZ et Jean-Charles RÉGIN. “Constructions and In-Place Operations for MDDs Based Constraints”. In : *Integration of AI and OR Techniques in Constraint Programming - 13th International Conference, CPAIOR 2016, Banff, AB, Canada, May 29 - June 1, 2016, Proceedings*. Sous la dir. de Claude-Guy QUIMPER. T. 9676.

Lecture Notes in Computer Science. Springer, 2016, p. 279-293. DOI : 10.1007/978-3-319-33954-2_20. URL : https://doi.org/10.1007/978-3-319-33954-2%5C_20.

Prud'homme et al. : Choco-solver : A Java library for constraint programming **Choco**

Charles PRUD'HOMME et Jean-Guillaume FAGES. "Choco-solver : A Java library for constraint programming". In : *Journal of Open Source Software* 7.78 (2022), p. 4708. DOI : 10.21105/joss.04708.

Truchet et al. : Musical constraint satisfaction problems solved with adaptive search **CTThese**

Charlotte TRUCHET et Philippe CODOGNET. "Musical constraint satisfaction problems solved with adaptive search". In : *Soft Comput.* 8.9 (2004), p. 633-640. DOI : 10.1007/S00500-004-0389-0. URL : <https://doi.org/10.1007/s00500-004-0389-0>.

6 Annexes

6.1 Notation Américaine

Definition 6.1 (*Notation Américaine*) Nous expliquerons ici la notation "américaine" car c'est celle qui est le plus proche d'une grammaire et donc implémentable aisément (pour du parsing), mais d'autres notations existent.

On donne la fondamentale avec une lettre $[A \rightarrow G]$ allant de La à Sol que l'on peut coupler avec une altération $\flat, \sharp$.

On ajoute une indication sur la tierce m pour mineur et M pour majeur, on peut aussi indiquer *sus2* (resp. *sus4*) dans le cas où l'on joue la seconde (resp. la quarte) à la place de la tierce.

On ajoute une indication pour savoir si la septième est présente avec $\mathcal{7}$, de même pour la neuvième et la treizième.

Il y a des cas exceptionnels :

- Accord diminué *dim*
- Accord avec la sixte *6*
- Accord avec la neuvième sans la septième *add9*
- Accord de dominante $\mathcal{7}$

Cette notation est presque identifiable à la grammaire suivante :

$$Acc := [A \rightarrow G] + [\flat|\sharp|\varepsilon] + [m|M|sus2|sus4|\varepsilon] + [5 - |5 + |\varepsilon] + [6|\varepsilon] + [7m|7M|7|\varepsilon] + [9|\varepsilon] + [11|\varepsilon] + [13|\varepsilon]$$

On a alors donné un ensemble de notes (sans hauteurs) et un accord effectif sera un ensemble de notes tel que ce soit exactement les notes données par la constructions avec la possibilité d'avoir des répétitions à des hauteurs différentes.

6.2 Exemple de transcription midi vers tablatures

Ici je rajoute des exemples de différentes exécutions du programme sur différentes suites d'accords avec le temps de calcul associé.